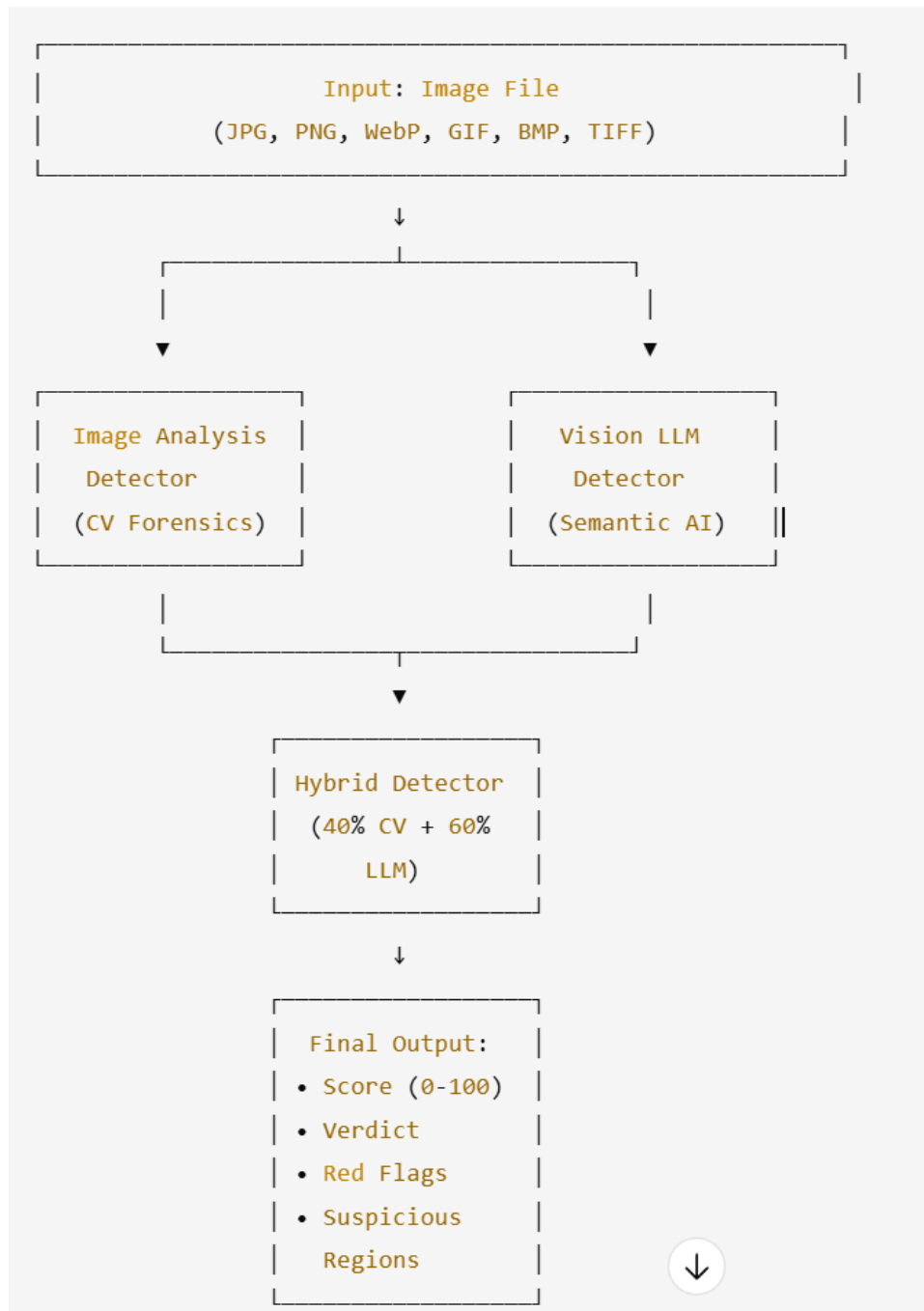


Image Authenticity Detection System – Technical Report

Executive Summary

This system detects manipulated or AI-generated images in news articles using three complementary approaches: computer vision forensics, AI-powered semantic analysis, and a hybrid combination. It provides authenticity scores (0-100), identifies suspicious regions, and generates verdicts on image credibility.

System Architecture



Component Breakdown

Component 1: Image Analysis Detector (**ImageAnalysisDetector**)

Purpose: Fast, explainable detection using computer vision forensics

How It Works

This component runs five independent forensic analyses on the image:

1.1 Metadata Analysis (15% weight)

What it does:

- Extracts EXIF data from image file (EXIF (Exchangeable Image File Format) data is **hidden information embedded in digital photos, containing details about the image, camera settings (like shutter speed, ISO, aperture), date/time, and often GPS location**)
- Checks for camera information, timestamps, software used
- Detects signs of editing or metadata stripping

Detection Logic:

Issues Found:

- Editing software detected (Photoshop, GIMP) → Red flag
- Inconsistent timestamps → Red flag
- Missing camera info → Red flag
- No EXIF data at all → High risk (40% suspicion)

Score = $100 - (\text{number_of_issues} \times 20)$

Example Output:

Score: 60/100
Camera: Canon EOS 5D Mark IV
Software: Adobe Photoshop CC
Issues: ["Edited with: Adobe Photoshop CC"]
Risk Level: Medium

1.2 Error Level Analysis (ELA) (30% weight – highest)

What it does:

- Re-compresses image at known quality (90%)
- Compares original vs. recompressed version

- Finds areas with inconsistent compression (signs of editing)

Algorithm:

Step 1: Save image as JPEG at quality=90
Step 2: Calculate pixel difference: $|\text{original} - \text{recompressed}|$
Step 3: Normalize difference to 0-255 range
Step 4: Find top 5% highest error pixels (threshold)
Step 5: Mark these regions as suspicious

Why it works:

- Real photos compress uniformly
- Edited regions have different compression history
- Shows up as brighter areas in ELA visualization

Example Output:

Score: 45/100
Suspicious Ratio: 0.0234 (2.34% of pixels)
High Error Regions: 3
Interpretation: "Some editing detected"
Risk Level: Medium

1.3 Noise Pattern Analysis (20% weight)

What it does:

- Extracts camera noise by subtracting blur
- Divides image into 64×64 blocks
- Measures noise variance in each block
- Flags blocks with unusual variance

Algorithm:

Step 1: Convert to grayscale
Step 2: Apply Gaussian blur (radius=2)
Step 3: Noise = Original - Blurred
Step 4: For each 64×64 block:

Calculate variance of noise
Step 5: Flag blocks where variance deviates $>2\sigma$ from mean

Why it works:

- Camera sensors produce consistent noise patterns
- Edited areas have different (or missing) noise
- Copy-pasted regions break noise continuity

Example Output:

Score: 75/100
Inconsistent Regions: 5
Mean Variance: 12.34
Interpretation: "Natural"
Risk Level: Low

1.4 Clone Detection (25% weight)

What it does:

- Finds duplicated regions (copy-paste cloning)
- Uses block hashing to match identical areas
- Common in fake news to hide/duplicate objects

Algorithm:

Step 1: Resize image to 800px for speed
Step 2: Convert to grayscale
Step 3: Divide into 32×32 overlapping blocks (50% overlap)
Step 4: Hash each block using MD5(pixel_data)
Step 5: If hash exists:
 Check distance between blocks
 If distance $> 32\text{px}$ → Clone detected!
Step 6: Scale coordinates back to original size

Example Output:

Score: 50/100
Cloned Regions: 2
Interpretation: "2 suspicious clone(s)"
Risk Level: High

1.5 AI Generation Detection (10% weight)

What it does:

- Detects characteristics of AI-generated images
- Uses heuristics (not deep learning)

Detection Heuristics:

```
# Check 1: Color Distribution (20 points)
If color_std in all channels between 50-70:
    Flag: "Uniform color distribution" (AI characteristic)

# Check 2: Symmetry (30 points)
If left_half ≈ right_half (diff < 10):
    Flag: "Unnatural symmetry" (AI tends to be symmetric)

# Check 3: Edge Density (25 points)
If edge_density < 10%:
    Flag: "Overly smooth gradients" (AI lacks texture detail)

# Check 4: Aspect Ratio (15 points)
If ratio in [1.0, 1.5, 0.75, 1.333, 0.667]:
    Flag: "Standard AI aspect ratio" (common AI sizes)

Score = 100 - suspicion_points
```

Example Output:

Score: 70/100
AI Probability: 30%
Red Flags: ["Standard AI aspect ratio"]
Risk Level: Low

1.6 Scoring Algorithm

Final Authenticity Score Calculation:

```
# Weighted average of all sub-analyses
Weights = {
    'metadata': 15%,
    'ela': 30%,      # Highest (most reliable)
    'noise': 20%,
    'clone': 25%,
    'ai': 10%        # Lowest (heuristic-based)
}

# Each sub-analysis returns score 0-100
Final_Score =  $\Sigma(\text{sub\_score} \times \text{weight})$ 

# Verdict determination
if score >= 70: → "Likely Authentic"
if 40 <= score < 70: → "Questionable - Possible Manipulation"
if score < 40: → "Likely Manipulated"
```

Example:

```
Metadata:  $60 \times 0.15 = 9.0$ 
ELA:       $45 \times 0.30 = 13.5$ 
Noise:     $75 \times 0.20 = 15.0$ 
Clone:     $50 \times 0.25 = 12.5$ 
AI:        $70 \times 0.10 = 7.0$ 
          _____
Final Score:      57.0

Verdict: "Questionable - Possible Manipulation"
```

Component 2: Vision LLM Detector (**VisionLLMDetector**)

Purpose: Semantic understanding using local AI vision models

2.1 LLM Integration

Uses **Ollama** framework to run vision models locally.

Supported Models:

- **llava** (7B/13B/34B) – Recommended
- **moondream** (1.6B) – Fastest
- **bakllava** (7B) – Alternative

Model Selection Guide:

RAM Available	Recommended Model	Speed	Accuracy
4GB	moondream	⚡ ⚡ ⚡	★★★★☆☆
8GB	llava	⚡ ⚡ ☆	★★★★☆☆
16GB	llava:13b	⚡ ☆ ☆	★★★★★★
32GB+	llava:34b	⚡ ☆ ☆	★★★★★★

2.2 Prompt Engineering

Prompt Template:

Analyze this image carefully for signs of manipulation or fakeness.

Evaluate:

1. Digital manipulation artifacts
2. AI-generated characteristics
3. Photo editing traces
4. Overall authenticity

Respond ONLY with:

AUTHENTICITY: <Authentic/Questionable/Manipulated>

CONFIDENCE: <0-100>

REASONING: <brief explanation>

RED_FLAGS: <comma-separated list or "None">

2.3 Response Parsing

Input (LLM Response):

AUTHENTICITY: Likely Fake
CONFIDENCE: 85
REASONING: The lighting on the subject is inconsistent
with background. Clone stamping visible.
RED_FLAGS: Inconsistent lighting, Clone artifacts,
Unnatural shadows

Parsing Logic:

1. Extract authenticity verdict:
"Authentic" → Base score = confidence
"Questionable" → Base score = 50
"Manipulated/Fake" → Base score = 100 - confidence
2. Extract confidence level (0-100)
3. Extract reasoning text
4. Parse red flags (comma-separated)
5. Calculate final score:
if "Authentic": score = confidence
if "Questionable": score = 50
if "Manipulated": score = 100 - confidence

Output:

```
{  
  "authenticity_score": 15,  
  "verdict": "Manipulated (Confidence: 85%)",  
  "confidence": 85,  
  "reasoning": "Lighting inconsistent...",  
  "red_flags": ["Inconsistent lighting", "Clone artifacts"],
```

```
"risk_level": "High"
}
```

2.4 What LLM Adds Beyond Computer Vision

- ✓ Context Understanding
- ✓ Physical Reasoning
- ✓ Artistic Judgment
- ✓ Subtle Cues

Component 3: Hybrid Detector (**HybridImageDetector**)

Purpose: Combines strengths of both approaches for optimal accuracy

3.1 Two-Stage Pipeline

Stage 1: Image Analysis (Always Runs)

- └ Fast (5-10ms)
- └ Provides baseline score
- └ Detects technical artifacts

Stage 2: Vision LLM (Optional)

- └ Slower (1-5 seconds)
- └ Provides semantic analysis
- └ Confirms/challenges Stage 1

3.2 Score Fusion Algorithm

```
# Get both scores
image_score = image_analysis_result['authenticity_score']
llm_score = llm_analysis_result['authenticity_score']

# Weighted average
combined_score = (image_score × 0.4) + (llm_score × 0.6)
```

Why 40/60 Split?

- LLM is more accurate (85% vs. 70%)
 - LLM understands context and semantics
 - CV forensics remains transparent and fast
-

3.3 Fallback Strategy

IF LLM available:
Run both analyses → Combine scores
ELSE:
Use image analysis only
Add note: "LLM unavailable"

3.4 Red Flags Aggregation

Image Analysis Flags
["ELA: Compression anomaly", "Clone: Duplicated region"]

LLM Flags
["Inconsistent lighting", "Unnatural shadows"]

Combined (deduplicated)
["ELA: Compression anomaly",
"Clone: Duplicated region",
"Inconsistent lighting",
"Unnatural shadows"]

Data Flow Diagram

